

# Hybrid-INoT: гибридная агентная архитектура с внутренней ролевой сегрегацией для инженерных задач в условиях длинного общего контекста

Даниил Привезенцев<sup>1\*</sup>

<sup>1\*</sup>Национальный исследовательский университет «Высшая школа экономики», Москва, Россия.

Corresponding author(s). E-mail(s): [daprivezentsev@edu.hse.ru](mailto:daprivezentsev@edu.hse.ru);

## Аннотация

Внешняя мультиагентная оркестрация повторно передаёт общий контекст отдельным ролям и увеличивает токенный расход. В работе предложена **Hybrid-INoT** — гибридная система, в которой planner, worker и critic разделяют один вызов модели, а верификация и инструменты остаются внешними. Проверка с сохранёнными артефактами сравнивает полный пакет В3 — объединённый вызов и детерминированный отбор контекста свыше 4096 токенов — с трёхвызовным В2 без компрессии. В реальном API-пилоте Gemini 3.1 Pro Preview ( $n = 5$  уникальных задач) В3 снизил средний токенный расход на 55.6–88.9% при целевых контекстах 256–16384 токена; независимая репликация Gemini 3.1 Flash-Lite ( $n = 20$ , seed 42) дала 49.6–90.3%. Средние внутрипарные приросты  $U_{\text{tok}}$  Flash-Lite равны 106.5, 144.8, 183.3 и 934.6%, причём каждая наблюдаемая пара превысила 15%; условный signed-rank анализ чувствительности прошёл поправку Холма ( $p = 9.54 \cdot 10^{-7}$ ). Четырёхточечные task-level OLS-сводки равны 3.425 для В2 и 0.219 для В3 токена на токен контекста (условный парный  $p = 1.91 \cdot 10^{-6}$ ); это не локальные производные через границу компрессии. Наблюдается эффект системного пакета, а не изолированной оркестрации. Обе архитектуры достигли pass@1 = 1.0; при нуле вредных исходов среди 20 уникальных пар iid-биномиальная планировочная граница равна 13.91 п.п., поэтому неухудшение в пределах 2 п.п. не доказано. Малая модель идеально согласуется с сохранёнными исходами HumanEval, но плохо — с приближённым SWE-bench Lite; эффект политики перезапуска не оценивался. Pro-пилот даёт \$429.96 экономии API-инференса на 10000 задач, а task-cluster ресэмплирование —

диапазон устойчивости \$417.33–\$442.90, но не популяционный интервал и не полный ТСО. Данные поддерживают механизм ресурсной эффективности в фиксированных выборках, оставляя открытыми сохранение качества, пользу маршрутизатора, вклад компонентов и перенос на реальные репозитории.

**Keywords:** Большие языковые модели, агентные системы, Introspection of Thought, токенизация, эффективность, экономика инференса, HumanEval, воспроизводимый эксперимент

## 1 Введение

Современный агент для программной инженерии должен последовательно понять постановку задачи, извлечь релевантный контекст из репозитория и построить изменение, которое затем можно проверить тестами или статическим анализом. На практике эти функции часто распределяются между несколькими ролями, причём под *ролью* понимается отдельный инстанс (экземпляр) большой языковой модели или отдельный ИИ-агент с собственной инструкцией и отдельным обращением к API: одна роль строит план, другая предлагает решение, третья критикует промежуточный результат, четвёртая запускает внешнюю проверку. Таким образом, каждая роль — это самостоятельный вызов LLM, а не абстрактная функция внутри одного вызова. Такое разбиение удобно концептуально, но становится затратным, когда все роли вынуждены заново читать один и тот же длинный контекст.

В настоящей работе рассматривается именно этот сценарий: задачи, в которых общими для всех ролей оказываются фрагменты кода, документация, сообщения компилятора и журналы выполнения. Нас интересует не только итоговое качество решения, но и цена выбранной схемы оркестрации в токенах, времени ответа и денежной стоимости вызовов. Поэтому далее статья сопоставляет классическую внешнюю мультиагентную организацию с гибридной схемой, в которой ролевая дискуссия частично переносится внутрь одного вызова модели.

### 1.1 Проблема многократной передачи контекста и оценка её значимости

В агентных системах на основе больших языковых моделей типичной конфигурацией является внешняя мультиагентная оркестрация (Classical-MAS): отдельные процессы или вызовы реализуют роли планировщика, исполнителя, критика и тестировщика [16–18]. В задачах программной инженерии, где несколько ролей читают одни и те же длинные фрагменты кода, логов и документации, такая схема обладает структурным дефектом: *каждая роль получает копию общего контекста  $C_0$  заново*, а между ролями дополнительно курсируют сообщения координации. Для количественной оценки этого эффекта введём следующие обозначения.

**Обозначения к формуле ниже.**

$\mathcal{A}$  — множество ролей в схеме Classical-MAS, каждая из которых реализована как отдельный вызов большой языковой модели;  $|\mathcal{A}|$  — число таких ролей (мощность множества).

$\mathcal{C}_0$  — исходный общий контекст задачи (фрагменты кода, документация, журналы выполнения), одинаковый для всех ролей;  $|\mathcal{C}_0|$  — его длина, измеряемая в токенах модели.

$\bar{h}$  — средняя длина одного координационного сообщения между ролями, также в токенах.

$D_{ctx}$  — суммарный токennyй расход на повторную передачу общего контекста всем ролям за одну задачу.

$H_{coord}$  — суммарный токennyй расход на координационные сообщения, которыми роли обмениваются в процессе решения задачи.

$\Theta(\cdot)$  — стандартная асимптотическая нотация: запись  $f = \Theta(g)$  означает, что  $f$  растёт не медленнее и не быстрее  $g$  с точностью до постоянных множителей.

В терминах введённых обозначений токennyй бюджет схемы Classical-MAS содержит слагаемое

$$D_{ctx} = \Theta(|\mathcal{A}| \cdot |\mathcal{C}_0|), \quad H_{coord} = O(|\mathcal{A}|^2 \cdot \bar{h}). \quad (1)$$

Смысл формулы (1) следующий. Первый член описывает линейный рост затрат на контекст, если каждая роль перечитывает  $\mathcal{C}_0$ . Второй член — зависящая от топологии верхняя асимптотическая оценка для случая обмена между всеми парами ролей, а не универсальная квадратичная нижняя граница. Поэтому эмпирическая проверка ниже опирается на непосредственно измеренную повторную передачу входа, а не на предположение о полном графе координации.

В выполненном API-пилоте этот структурный эффект наблюдался непосредственно. При целевом контексте 16 384 токена Classical-MAS расходовал в среднем 59 631 токен на задачу, из которых 57 485 (96.4%) приходились на вход, тогда как Hybrid-INoT расходовал 6 608 токенов. Стоимость одного случая составила соответственно \$0.14072 и \$0.02949 при зафиксированных ценах Gemini 3.1 Pro Preview [35]. Эти значения относятся к первым пяти задачам HumanEval и не должны переноситься на иной состав задач без повторного измерения.

Таким образом, актуальность проблемы определяется проверяемой величиной: в исследованном длинноконтекстном режиме повторная передача входа дала почти девятикратную разницу в суммарных токенах между B2 и B3. Далее теоретическая модель отделяется от эмпирического свидетельства, а область выводов ограничивается фактически выполненными пилотами.

## 1.2 Существующие подходы и их ограничения

Ближайшее поле шире дихотомии «саморефлексия против MAS»: внутримодельное уточнение [7, 8], агенты рассуждение–действие [6, 21], многоперсонное prompting одной LLM [9], ролевая кооперация через общую доску [10], разреженный внешний дебат [11] и намеренно простой ремонт репозитория Agentless [12]. Hybrid-INoT непосредственно развивает INoT [1]. Поэтому заявленная новизна узка: операциональная интеграция внутреннего вызова planner–worker–critic,

детерминированного отбора контекста, внешней проверки, маршрутизатора развёртывания и аудируемого учёта ресурсов. Работа не претендует на изобретение ролевого prompting, самокритики, компрессии или инструментальной проверки по отдельности.

### 1.3 Основные результаты и вклад статьи

В статье получены следующие результаты:

- C1.** Операциональное определение Hybrid-INoT с явной сигнатурой состояний, функцией маршрутизации `RouteMode` и условием завершения внутреннего ролевого цикла (§4).
- C2.** Утверждение 2, задающее достаточное балансовое условие преимущества полного пакета Hybrid-INoT по токенам (§5).
- C3.** Неравенство (11), задающее условие экономической эффективности малой модели самопроверки (§5).
- C4.** Система метрик токеной эффективности, стоимости и поддерживаемости кода с обоснованной формой агрегации (§6).
- C5.** Эмпирическая проверка с сохранёнными артефактами реальных API-вызовов: масштабирование B2 и B3, абляции, разведочная screening-диагностика и межмодельное сравнение E6 (§7).
- C6.** Согласованные с данными вердикты: полная составная H1 не доказана, H2 причинно не оценивалась, H3 не поддержана прямым сравнением (§7).

Исходный код стенда, конфигурации, сырые результаты и скрипт построения таблиц и графиков опубликованы в отдельном репозитории и зафиксированы по commit hash (§7).

## 2 Связанные работы и отличие предлагаемого подхода

### 2.1 От chain-of-thought к внутреннему ролевому дебату

В работе Wei et al. [5] показано, что явная вербализация промежуточных шагов рассуждения может повышать качество решения на задачах, требующих нескольких связанных логических переходов. В данном контексте под промежуточными шагами понимаются текстовые фрагменты, с помощью которых модель раскладывает сложный вывод на последовательность локальных подзадач, а под качеством — итоговая успешность решения целевой задачи. Подход Self-Refine [7] формализует цикл «генератор–обратная связь–исправление» внутри одной модели; Reflexion [8] дополняет такую схему механизмом рефлексивной памяти, а ReAct [6] связывает внутреннее рассуждение с обращениями к внешним действиям и инструментам. Работа Sun и Zeng [1] описывает Introspection of Thought (INoT) как структурированный внутренний дебат нескольких ролей. В то же время Turpin et al. [13] показывают, что промежуточный текст цепочки не следует интерпретировать как достоверное описание скрытого вычислительного процесса.

Поэтому в настоящей работе интроспекция рассматривается прежде всего как вычислительный механизм отбора кандидатов, критики и селективного перезапуска, а не как буквальное окно в скрытое состояние модели.

## 2.2 Мультиагентные системы для программной инженерии

Weiss [14] и Wooldridge [15] задают теоретическую рамку MAS. MetaGPT [16] и ChatDev [17] переносят её в контекст больших языковых моделей: роли реализуются как отдельные вызовы, каждая получает собственную копию контекста. Обзорные работы указывают на два ключевых ограничения Classical-MAS: распространение ошибки по цепочке и рост стоимости координации [18, 19]. SWE-bench и SWE-agent [20, 21] показывают, что практическая автоматизация задач программной инженерии требует длинного контекста и итеративного редактирования, что ещё сильнее обостряет проблему ретрансмиссии  $D_{ctx}$ .

Single-Persona Prompting (SPP) моделирует несколько персон через многотуповое prompting одной LLM [9]; Self-Collaboration Code Generation координирует специализированные роли через общую доску и языкового тестировщика [10]; S2-MAD сокращает граф коммуникаций внешнего дебата [11]. Agentless показывает, что компактный конвейер локализации, ремонта и проверки может быть сильной базой без сложного общества агентов [12]. Эти результаты требуют сравнивать не только названия архитектур, но и топологию вызовов и вклад каждого компонента.

## 2.3 Компрессия контекста и внешняя верификация

LongLLMLingua и LLMLingua-2 [24, 25] демонстрируют, что компрессия промпта сохраняет или улучшает качество на длинном контексте при снижении стоимости. Jia et al. [26] применяют этот подход к коду. Для самопроверки кода недостаточно языковой рефлексии: необходимы внешние наблюдаемые сигналы — тесты, статический анализ и сообщения компилятора [27–29].

## 2.4 Сопоставление Hybrid-INoT с предшественниками

Для сопоставления ближайших подходов выделим ролевой механизм, топологию вызовов модели, внешнюю проверку, маршрутизацию и явный учёт ресурсов.

Таблица 1 показывает значительное пересечение на уровне компонентов. Проверяемый вклад настоящей работы — интегрированный и инструментированный пакет, а не безусловный приоритет архитектурной идеи.

# 3 Формальная постановка задачи

## 3.1 Задача

Введём базовые множества, используемые в формальной постановке задачи. Их содержательный смысл — описать, с какими входными и выходными данными работает агент и в каком пространстве задана его случайность.

**Таблица 1** Сравнение Hybrid-INoT с близкими архитектурными классами

| Метод                      | Роли / вызовы                              | Внешняя проверка             | Маршрутизация         | Учёт стоимости                                |
|----------------------------|--------------------------------------------|------------------------------|-----------------------|-----------------------------------------------|
| Self-Refine / Reflexion    | Итеративная самокритика; повторные вызовы. | Необязательна.               | Фиксированная.        | Нет учёта контекстного склона.                |
| SPP                        | Несколько персон одной LLM; много туров.   | Не центральна.               | Фиксированная.        | Сокращение вызовов не является estimand.      |
| Self-Collaboration         | Роли и общая доска; несколько вызовов.     | Языковой тестировщик.        | Фиксированная.        | Без модели контекстного склона.               |
| MetaGPT / ChatDev / S2-MAD | Внешние агенты или разреженный дебат.      | Зависит от конвейера.        | Фиксированный граф.   | Системная цена.                               |
| Agentless                  | Заданные стадии; мало вызовов.             | Проверка патча.              | Фиксированные стадии. | Простота и цена ремонта как baseline.         |
| Hybrid-INoT                | Planner-worker-critic в одном вызове.      | Обязательна после генерации. | Флаги tool/parallel.  | Токенный баланс, API-цена, контекстный склон. |

$\mathcal{X}$  — множество *текстовых спецификаций задач*. Каждый элемент  $x \in \mathcal{X}$  — это строка естественного языка (например, сформулированная пользователем задача или формулировка задачи в HumanEval).

$\mathcal{C}$  — множество *контекстных артефактов*. Каждый элемент — это конечная последовательность текстовых фрагментов: файлов кода, кусков документации, сообщений компилятора, журналов выполнения. Конкретный контекст задачи  $C_0 \in \mathcal{C}$  — это набор именно таких артефактов, имеющих отношение к задаче.

$\mathcal{Y}$  — множество *кандидатных решений*. Каждый элемент  $y \in \mathcal{Y}$  — это текстовый артефакт, например исходный код функции или патч в формате **unified diff**, который затем подаётся во внешнюю проверяемую функцию (запуск тестов, статический анализ). Таким образом,  $y$  — не числовой вектор, а именно текст.

$(\Omega, \mathcal{F}, \mathbb{P})$  — *вероятностное пространство*, на котором определены все случайные величины задачи. Здесь  $\Omega$  — множество возможных исходов одного запуска агента (все возможные сценарии развития процесса решения),  $\mathcal{F}$  —  $\sigma$ -алгебра наблюдаемых событий над  $\Omega$  (формальная структура, позволяющая корректно говорить о вероятностях событий),  $\mathbb{P}$  — вероятностная мера на  $\mathcal{F}$ , учитывающая сразу три источника случайности: стохастическую генерацию токенов в LLM при температуре  $> 0$ , выбор начального числа генератора псевдослучайных чисел (*seed* — целое число, инициализирующее детерминированную последовательность псевдослучайных чисел; фиксация *seed* обеспечивает воспроизводимость запуска) и недетерминизм внешних инструментальных вызовов (например, таймауты тестов). Проще говоря,  $(\Omega, \mathcal{F}, \mathbb{P})$  — это математический способ записать утверждение «запуск агента — это случайный эксперимент».

**Определение 1** (Агентная инженерная задача). *Агентная инженерная задача — это кортеж  $(x, C_0, \mathcal{Z}, \mathcal{V})$ , где  $x \in \mathcal{X}$  — текстовое описание задания;*

$C_0 \in \mathcal{C}$  — исходный контекст задачи, включающий код, документацию и журналы выполнения;  $\mathcal{Z}$  — конечное множество разрешённых внешних инструментов (запуск тестов, линтер, компилятор, поиск по репозиторию и т. п.);  $\mathcal{V}: \mathcal{Y} \rightarrow \{0, 1\}$  — внешняя проверяемая функция успеха, возвращающая 1 при корректном решении (например, при прохождении всего тестового набора) и 0 иначе.

### **Случайные величины, индуцируемые стратегией агента.**

Стратегия агента — это зафиксированная процедура, отображающая описание задачи  $x$  и контекст  $C_0$  в распределение на  $\mathcal{Y}$ . Она индуцирует следующие случайные величины (функции из  $\Omega$  в числовые множества):

$Y: \Omega \rightarrow \mathcal{Y}$  — случайный выход агента: итоговое кандидатное решение, которое агент возвращает в конце работы на конкретном запуске.

$T: \Omega \rightarrow \mathbb{R}_{\geq 0}$  — суммарный токeнный расход на одну задачу. Значение  $T(\omega)$  — общее число токeнов (суммарно по всем вызовам модели во всех ролях и итерациях), израсходованных на решение задачи в конкретном запуске  $\omega$ . Здесь  $\mathbb{R}_{\geq 0}$  — множество неотрицательных действительных чисел.

$C_{inf}: \Omega \rightarrow \mathbb{R}_{\geq 0}$  — денежная стоимость инференса на одну задачу (в валюте расчёта, например в долларах США), вычисляемая как произведение числа входных и выходных токeнов на соответствующие тарифы API.

$\ell: \Omega \rightarrow \mathbb{R}_{\geq 0}$  — задержка выполнения одной задачи (wall-clock time в секундах) от получения запроса до возврата решения.

Случайность этих величин обусловлена стохастической генерацией LLM и недетерминизмом внешних инструментов; математическое ожидание  $\mathbb{E}[\cdot]$  во всех последующих выражениях берётся по мере  $\mathbb{P}$  на  $(\Omega, \mathcal{F})$ .

### **Целевая функция и ограничения.**

Цель агента — максимизировать ожидаемое качество решения, удерживая ожидаемые затраты в пределах заданных бюджетов. Формально:

$$\max \mathbb{E}[\mathcal{V}(Y)] \quad \text{при условиях} \quad \mathbb{E}[T] \leq B, \quad \mathbb{E}[C_{inf}] \leq B_{\$}, \quad \mathbb{E}[\ell] \leq B_{\ell}. \quad (2)$$

### **Расшифровка всех величин в (2):**

$\mathbb{E}[\mathcal{V}(Y)]$  — математическое ожидание функции успеха  $\mathcal{V}$  от случайного выхода  $Y$ . Поскольку  $\mathcal{V} \in \{0, 1\}$ , эта величина равна вероятности того, что агент на случайном запуске вернёт корректное решение (то есть  $\Pr[\mathcal{V}(Y) = 1]$ ).

$\mathbb{E}[T]$  — среднее число токeнов, расходуемых агентом на одну задачу (по распределению, заданному  $\mathbb{P}$ ).

$\mathbb{E}[C_{inf}]$  — средняя денежная стоимость инференса на одну задачу.

$\mathbb{E}[\ell]$  — средняя задержка ответа на одну задачу.

$B, B_{\$}, B_{\ell}$  — верхние бюджетные лимиты, заданные пользователем или эксплуатационным контекстом:  $B$  — допустимый средний токeнный расход (в токeнах),  $B_{\$}$  — допустимая средняя стоимость (в валюте расчёта),  $B_{\ell}$  — допустимая средняя задержка (в секундах).

Словами: формула (2) означает «среди стратегий, укладывающихся в заданные бюджеты по токенам, деньгам и времени, выбрать ту, у которой наибольшая ожидаемая доля успешных решений».

### Сводка обозначений

Ниже собраны все обозначения, используемые в дальнейшем тексте, с явными определениями. Часть из них введена выше при постановке задачи; остальные появятся в разделах об архитектуре и аналитической модели.

$\mathcal{C}_0 \in \mathcal{C}$  — исходный общий контекст задачи (конечная последовательность текстовых артефактов);  $|\mathcal{C}_0| \in \mathbb{N}$  — его длина в токенах модели.

$\hat{\mathcal{C}}$  — сжатая версия исходного контекста, используемая во внутреннем цикле Hybrid-INoT;  $|\hat{\mathcal{C}}| \leq |\mathcal{C}_0|$ .

$\mathcal{A}$  — конечное множество внешних ролей в схеме Classical-MAS, каждая из которых — отдельный вызов LLM;  $|\mathcal{A}| \in \mathbb{N}$  — число таких ролей.

$Y: \Omega \rightarrow \mathcal{Y}$  — случайный выход агента;  $y \in \mathcal{Y}$  — конкретный (реализованный) кандидат решения.

$K \in \mathbb{N}$  — число кандидатов, одновременно порождаемых ролью  $r_{\text{work}}$  на одной итерации внутреннего цикла (базовая конфигурация:  $K = 3$ ).

$T: \Omega \rightarrow \mathbb{R}_{\geq 0}$  — случайная величина суммарного токенового расхода на одну задачу;  $\mathbb{E}[T] = \int_{\Omega} T(\omega) d\mathbb{P}(\omega)$  — её математическое ожидание, вычисляемое по мере  $\mathbb{P}$  на вероятностном пространстве  $(\Omega, \mathcal{F}, \mathbb{P})$ , введённом выше.

$U_{\text{tok}} = Q/T$  — показатель токеновой эффективности: отношение агрегированной оценки качества  $Q$  (см. §6) к суммарному токеновому расходу; единица измерения — «качество на 1000 токенов».

$\tau^* \in \mathbb{R}_{\geq 0}$  — теоретическая точка пересечения, следующая только из достаточных балансовых предположений; эксперимент ЕЗ её не идентифицирует.

$\delta_{H1} \in (0, 1)$  — минимальный практически значимый относительный прирост показателя  $U_{\text{tok}}$ , используемый в гипотезе H1 (базовое значение  $\delta_{H1} = 0,15$ , то есть 15%).

$C_{\text{check}}^S \in \mathbb{R}_{\geq 0}$  — денежная стоимость одной предварительной проверки решения, выполняемой *малой* (дешёвой) моделью самопроверки; верхний индекс  $S$  обозначает small.

$C_{\text{rerun}}^L \in \mathbb{R}_{\geq 0}$  — денежная стоимость одного повторного запуска *крупной* (дорогой) модели; верхний индекс  $L$  обозначает large.

$\rho_0, \rho_1 \in [0, 1]$  — вероятности того, что потребуются дорогостоящий повторный запуск крупной модели:  $\rho_0$  — без предварительной проверки малой моделью,  $\rho_1$  — при наличии такой проверки; по построению  $\rho_1 \leq \rho_0$ .

$\mathbb{E}[\cdot]$  — математическое ожидание на вероятностном пространстве  $(\Omega, \mathcal{F}, \mathbb{P})$ .

$\mathbb{I}[\cdot]$  — индикаторная функция:  $\mathbb{I}[A] = 1$ , если условие  $A$  истинно, и 0 иначе.

## 3.2 Гипотезы исследования

Перед формулировкой гипотез поясним несколько обозначений и статистических процедур, встречающихся ниже, чтобы избавить читателя от необходимости искать их на стороне.



«п.п.» (процентные пункты) — единица разности двух величин, выраженных в процентах. Если одна конфигурация даёт  $\text{pass}@1 = 68\%$ , а другая =  $66\%$ , то разница составляет 2 процентных пункта ( $68 - 66 = 2$ ), но при этом относительное изменение равно  $\approx 3\%$ . В тексте всегда используется «п.п.» именно в этом смысле: абсолютная разность процентов, а не отношение.

*Парный критерий Вилкоксона (Wilcoxon signed-rank test)* — непараметрический статистический тест для сравнения двух связанных выборок (например, значений метрики на одних и тех же задачах при двух конфигурациях). В отличие от  $t$ -критерия не требует нормальности распределения; проверяет гипотезу о симметрии распределения разностей относительно нуля. Применяется стандартно при небольших и негауссовских выборках [32].

*Поправка Холма–Бонферрони (Holm–Bonferroni correction)* — пошаговая процедура корректировки уровня значимости при множественном сравнении. Если одновременно проверяется  $m$  гипотез с  $p$ -значениями  $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$ , то  $k$ -я гипотеза отвергается, если  $p_{(k)} \leq \alpha/(m - k + 1)$ . Эта поправка контролирует вероятность хотя бы одной ложной находки (family-wise error rate) при  $\alpha = 0,05$  и является менее консервативной, чем обычная поправка Бонферрони [33].

*task-resampling диапазон устойчивости* — квантили 2,5% и 97,5% статистики после  $10^4$  ресэмплирования наблюдаемых задач с возвращением. Для первых неслучайно выбранных задач этот диапазон показывает чувствительность к их перевзвешиванию, но не является design-based доверительным интервалом для популяции.

*seed (начальное число генератора псевдослучайных чисел)* — целое число, которым инициализируется локальная псевдослучайная последовательность и передаётся поддерживающему этот параметр API. Фиксация seed обеспечивает повторяемость локальной выборки, но сама по себе не гарантирует битовую воспроизводимость закрытой LLM.

*Synthetic Tool-Use Suite* — контролируемый микротест стенда для сравнения последовательного и параллельного выполнения независимых инструментальных задержек. Он не является внешним бенчмарком и используется только как проверка механизма НЗ; точная реализация находится в сопроводительном репозитории.

**Н1** (*преимущество при длинном контексте*). Существует пороговая длина исходного контекста  $\tau^*$  такая, что при  $|\mathcal{C}_0| > \tau^*$  архитектура Hybrid-INoT обеспечивает рост показателя  $U_{\text{tok}}$  не менее чем на  $\delta_{H1} = 15\%$  по сравнению с Classical-MAS, при этом качество решения ( $\text{pass}@1$ ) не ухудшается более чем на 2 п.п.

*Проверка:* ЕЗ (§7.2) сообщает среднее по конечному набору и число пар выше порога; критерий Вилкоксона, task-resampling диапазоны и поправка Холма являются условным анализом чувствительности, а не design-based выводом о популяции.

**Н2** (*экономическая оправданность малой модели самопроверки*). Существует режим, в котором предварительная проверка малой моделью уменьшает вероятность дорогостоящего повторного запуска настолько, что выполняется условие (11):  $C_{\text{check}}^S < (\rho_0 - \rho_1) C_{\text{run}}^L$ .

*Предусмотренная проверка:* интегрированный policy-эксперимент с per-case решениями о перезапуске и затратами. Сохранённый E4 (§7.4) этому дизайну не соответствует и используется только как разведочная диагностика screening.

**ИЗ (граница применимости).** При задачах с истинно независимыми подзадачами и параллельными инструментами преимущество Hybrid-INoT по времени ответа исчезает или становится отрицательным.

*Проверка:* архитектурное сравнение на пяти задачах и отдельный контролируемый микротест последовательных/параллельных инструментов E2c (§7.3).

В H1 внутри пары фиксированы модель, инструменты, верификация, задача, seed, температура и промпт задачи. Однако treatment является *системным пакетом*: B2 использует три вызова над несжатым контекстом, а B3 объединяет ролевые префиксы в одном вызове и применяет детерминированный отбор свыше 4096 токенов. Поэтому E3 оценивает эффект пакета, а не изолированный причинный вклад оркестрации или компрессии. Порог  $\delta_{H1} = 15\%$  — минимально практически значимый эффект; экономия ресурсов не подменяет полный составной критерий H1.

## 4 Метод: Hybrid-INoT

### 4.1 Определение метода и базовая идея

Прежде чем переходить к формальному описанию, дадим содержательное определение метода, на котором строится предлагаемая архитектура, и сформулируем саму идею Hybrid-INoT в минимально необходимой форме.

**Определение 2** (Introspection of Thought, INoT). Introspection of Thought (INoT) — это приём организации рассуждений большой языковой модели, при котором внутри одного обращения к модели разворачивается структурированный диалог нескольких ролей (ролевых префиксов: «планировщик», «исполнитель», «критик» и т. п.). Каждая роль читает и дописывает общий рабочий буфер в рамках одного прогона модели, что позволяет удерживать общий контекст без дублирования [1]. В отличие от внешней мультиагентной оркестрации, каждая роль INoT не является отдельным вызовом LLM — это ролевой префикс внутри единственного вызова.

**Определение 3** (Hybrid-INoT). Hybrid-INoT — архитектура, предлагаемая в настоящей работе, которая комбинирует внутренний ролевой цикл planner–worker–critic в смысле Определения 2 с обязательной внешней инструментальной проверкой и предобработкой контекста. А именно:

- роли планирования ( $r_{\text{plan}}$ ), генерации кандидатов ( $r_{\text{work}}$ ) и критики ( $r_{\text{crit}}$ ) выполняются как ролевые префиксы внутри одного вызова крупной LLM над сжатым общим контекстом  $\hat{C}$ ;
- вовне вынесены три и только три класса операций: (i) компрессия контекста  $C_0 \mapsto \hat{C}$  (отдельным лёгким модулем), (ii) внешняя проверяемая верификация  $r_{\text{self}}$  (запуск тестов, статического анализа, компилятора), (iii) функция

маршрутизации *RouteMode*, решающая, какой режим работы применить к данной задаче.

Таким образом, *Hybrid-INoT* — это промежуточная конфигурация между чисто внутриконтекстной самокритикой (*Self-Refine*, *Reflexion*) и полностью внешней мультиагентной оркестрацией (*MetaGPT*, *ChatDev*), у которой внутри сохраняется разделяемый контекст и ролевая сегрегация, а снаружи остаются только операции с проверяемыми, инструментальными сигналами.

Дальнейшие подпункты формализуют эту идею: §4.2 — состояния и сигнатуры ролей, §4.3 — функция маршрутизации, §4.4 — внутренний цикл и условие его завершения, §4.5 — компрессия контекста, §4.6 — селективный перезапуск.

## 4.2 Состояния и сигнатуры ролей

Определим состояние системы  $\sigma = (x, \hat{\mathcal{C}}, \mathcal{H}, i)$ , где  $x \in \mathcal{X}$  — текстовое описание задачи (см. Определение 1),  $\hat{\mathcal{C}}$  — сжатый общий контекст,  $\mathcal{H}$  — история итераций,  $i \in \{0, \dots, I\}$  — счётчик итераций, а  $I \in \mathbb{N}$  — верхний предел числа итераций. Отдельно общий API-клиент ведёт денежный счётчик эксперимента и отклоняет запрос, если его оценочная стоимость пересечёт настроенный жёсткий лимит.

Пусть  $\Pi$  — *пространство планов*, то есть множество возможных текстовых планов решения (в базовой реализации — короткий маркированный список шагов на естественном языке). Пусть  $K \in \mathbb{N}$  — *число кандидатных решений*, одновременно порождаемых ролью  $r_{\text{work}}$  на одной итерации внутреннего цикла; в базовой конфигурации  $K = 3$ . Тогда каждая роль задаётся сигнатурой  $r: \mathcal{S}_r^{\text{in}} \rightarrow \mathcal{S}_r^{\text{out}}$ :

$$r_{\text{plan}}: (x, \hat{\mathcal{C}}) \mapsto \pi \in \Pi, \quad (3)$$

$$r_{\text{work}}: (\pi, \hat{\mathcal{C}}, K) \mapsto \mathbf{y} = (y_1, \dots, y_K) \in \mathcal{Y}^K, \quad (4)$$

$$r_{\text{crit}}: (\mathbf{y}, \hat{\mathcal{C}}) \mapsto (s_1, \dots, s_K) \in [0, 1]^K, \quad (5)$$

$$r_{\text{self}}: (y \in \mathcal{Y}) \mapsto (v, M, e) \in \{0, 1\} \times [0, 1] \times \mathcal{E}. \quad (6)$$

### Расшифровка сигнатур (3)–(6):

$r_{\text{plan}}$  — роль *планировщика*. Входы: текст задачи  $x$  и сжатый контекст  $\hat{\mathcal{C}}$ . Выход: план  $\pi \in \Pi$  — последовательность шагов решения.

$r_{\text{work}}$  — роль *исполнителя* (генератора кандидатов). Входы: план  $\pi$ , сжатый контекст  $\hat{\mathcal{C}}$ , число требуемых кандидатов  $K$ . Выход: набор  $\mathbf{y} = (y_1, \dots, y_K)$  из  $K$  кандидатных решений  $y_k \in \mathcal{Y}$ ; запись  $\mathcal{Y}^K$  —  $K$ -я декартова степень множества  $\mathcal{Y}$ . Значение  $K$  задаётся гиперпараметром архитектуры (в базовой конфигурации  $K = 3$ ) и регулирует разнообразие: при  $K = 1$  критик всегда оценивает единственный вариант и роль  $r_{\text{crit}}$  вырождается; увеличение  $K$  повышает вероятность получить хоть одно корректное решение ценой пропорционального роста выходных токенов.

$r_{crit}$  — роль критика. Входы: набор кандидатов  $\mathbf{y}$  и контекст  $\hat{\mathcal{C}}$ . Выход: набор скалярных оценок  $(s_1, \dots, s_K) \in [0, 1]^K$ , где  $s_k$  — оценка качества  $k$ -го кандидата (1 — безусловно принять, 0 — безусловно отвергнуть).

$r_{self}$  — внешний процесс самопроверки: запуск тестов, статического анализа и линтера. Вход: один отобранный кандидат  $y$ . Выход: тройка  $(v, M, e)$ , где  $v \in \{0, 1\}$  — бинарный вердикт,  $M \in [0, 1]$  — оценка поддерживаемости (см. формулу (16)), а  $e \in \mathcal{E}$  — подтверждённое инструментами сообщение об ошибке.

Роли  $r_{plan}, r_{work}, r_{crit}$  реализуются как ролевые префиксы внутри *одного* вызова крупной модели над общим  $\hat{\mathcal{C}}$ ; роль  $r_{self}$  — отдельный внешний процесс (запуск тестов, линтера, статического анализатора). Содержательный смысл сигнатур: внутренний цикл вырабатывает план, по нему генерирует  $K$  кандидатов, оценивает их критиком и передаёт лучшего на внешнюю инструментальную проверку, которая возвращает единственный объективный сигнал об успехе.

### 4.3 Функция маршрутизации

*Содержательный смысл.*

RouteMode выбирает внутренний либо внешний рабочий процесс по двум дешёвым признакам задачи: обязательности внешнего инструментального workflow и доминирующему параллелизму подзадач. Ниже приведено правило, реально присутствующее в зафиксированном commit.

**Определение 4** (RouteMode). Пусть  $\varphi_{tool}(x) \in \{0, 1\}$  считывает Boolean-поле *metadata requires\_tool*, а  $\varphi_{par}(x) \in \{0, 1\}$  равно единице, когда поле *metadata parallelism* не меньше двух. Тогда

$$RouteMode(x) = \begin{cases} \text{external}, & \varphi_{tool}(x) = 1 \vee \varphi_{par}(x) = 1, \\ \text{internal}, & \text{иначе.} \end{cases}$$

Зафиксированный код не выводит эти флаги из произвольной инструкции: для синтетических tool-задач их задаёт загрузчик benchmark. Обучаемый или инструкционный metadata-классификатор и порог по длине контекста возможны как будущие расширения, но в исследуемом commit их нет. В ЕЗ внутренний режим ВЗ принудительно используется во всех ячейках, поэтому качество маршрутизатора и любой порог маршрутизации не оцениваются.

### 4.4 Внутренний цикл и условие завершения

**Утверждение 1** (Терминация внутреннего цикла). Пусть каждая операция внутри одной итерации алгоритма 1 завершается за конечное время, а число итераций ограничено сверху параметром  $I$ . Тогда алгоритм завершится не позднее чем после  $I$  итераций: он либо вернёт решение после успешной внешней проверки, либо после итерации  $i > 1$  остановится при  $s_{i,k^*} - s^{prev} < \varepsilon$ , либо после исчерпания лимита итераций вернёт лучший найденный кандидат.

---

**Algorithm 1** Hybrid-INoT: внутренний интроспективный цикл

---

**Require:** задача  $x$ , контекст  $\mathcal{C}_0$ , максимум итераций  $I$ , число кандидатов  $K$ , лимит перезапусков  $R$

**Ensure:** решение  $y^*$ , флаг  $v^*$ , оценка  $M^*$

```
1:  $\mathcal{C} \leftarrow \text{PolicySanitize}(x, \mathcal{C}_0)$ 
2:  $\hat{\mathcal{C}} \leftarrow \text{CompressIfNeeded}(\mathcal{C})$  ▷ Алг. 2
3:  $m \leftarrow \text{RouteMode}(x)$  ▷ Опр. 4
4: if  $m = \text{external}$  then return  $\text{ExternalAgentWorkflow}(x, \hat{\mathcal{C}})$ 
5: end if
6:  $s^{\text{best}} \leftarrow -\infty$ ;  $s^{\text{prev}} \leftarrow -\infty$ ;  $y^{\text{best}} \leftarrow \perp$ ;  $r \leftarrow 0$ ;  $e \leftarrow \emptyset$ 
7: for  $i = 1$  to  $I$  do
8:    $(\pi_i, \{y_{i,k}\}, \{s_{i,k}\}) \leftarrow \text{CombinedLargeModelCall}(x, \hat{\mathcal{C}}, K, e)$  ▷ один API-вызов
   с ролевыми префиксами
9:    $k^* \leftarrow \arg \max_k s_{i,k}$ ;  $y_i \leftarrow y_{i,k^*}$ 
10:  if  $s_{i,k^*} > s^{\text{best}}$  then  $s^{\text{best}} \leftarrow s_{i,k^*}$ ;  $y^{\text{best}} \leftarrow y_i$ 
11:  end if
12:   $(v_i, M_i, e_i) \leftarrow r_{\text{self}}(y_i)$  ▷  $e_i$ : подтверждённый вывод инструментов
13:  if  $v_i = 1$  then return  $(y_i, 1, M_i)$ 
14:  end if
15:   $e \leftarrow \text{Truncate}(e_i, 600)$ 
16:  if  $r < R$  then
17:     $(y'_i, v'_i, M'_i, e'_i) \leftarrow \text{SelectiveRerun}(x, \{y_{i,k}\}, \{s_{i,k}\})$ ;  $r \leftarrow r + 1$ 
18:    if  $v'_i = 1$  then return  $(y'_i, 1, M'_i)$ 
19:    end if
20:  end if
21:  if  $i > 1 \wedge s_{i,k^*} - s^{\text{prev}} < \varepsilon$  then break ▷ см. УТВ. 1
22:  end if
23:   $s^{\text{prev}} \leftarrow s_{i,k^*}$ 
24: end for
25: return  $(y^{\text{best}}, 0, M(y^{\text{best}}))$ 
```

---

*Доказательство* Внешний цикл по индексу  $i$  содержит не более  $I$  проходов. Каждая итерация включает конечный набор шагов: построение плана, генерацию  $K$  кандидатов, критическую оценку, внешнюю проверку и обновление контекста. По предположению все эти операции завершаются за конечное время, следовательно, завершается и любая отдельная итерация. Поэтому алгоритм либо прекращается досрочно по инструкции **return** или **break**, либо после  $I$ -й итерации достигает последней инструкции **return**. Тем самым время работы внутреннего цикла сверху ограничено конечным числом итераций, а утверждение доказано.  $\square$

### Замечание

Утверждение 1 гарантирует только завершение алгоритма, но не монотонный рост оценок  $s_i$  и не корректность самого критика. Сценарий, в котором критик систематически подтверждает неверные решения, рассматривается отдельно в E2b (§7.3) как одна из угроз валидности.

## 4.5 Компрессия контекста

---

### Algorithm 2 Компрессия длинного контекста

---

**Require:**  $\mathcal{C}_0$ , порог длины  $\tau_{\text{len}}$

**Ensure:**  $\hat{\mathcal{C}}$

- 1: **if**  $|\mathcal{C}_0| \leq \tau_{\text{len}}$  **then return**  $\mathcal{C}_0$
  - 2: **end if**
  - 3: разбить  $\mathcal{C}_0$  на текстовые блоки  $\{b_j\}$  и выделить идентификаторы задачи
  - 4:  $\text{rel}(b_j) \leftarrow 5 r_{\text{id}}(b_j) + 2 \mathbb{I}_{\text{sig}}(b_j) + 4 \mathbb{I}_{\text{err}}(b_j) - 0.001 |b_j|$
  - 5: сохранить блоки сигнатур/ошибок; остальные добавлять по убыванию  $\text{rel}(b_j)$
  - 6: **return** блоки, сумма гроху-счётчиков которых не превышает  $\tau_{\text{len}} = 4096$
- 

Здесь  $r_{\text{id}}$  — доля идентификаторов задачи в блоке, а  $\mathbb{I}_{\text{sig}}$  и  $\mathbb{I}_{\text{err}}$  отмечают сигнатуры и тексты ошибок/логов. Реализация ограничивает сумму гроху-счётчиков блоков, но затем соединяет их разделителями; поэтому сериализованный контекст может немного превысить 4096 гроху-токенов (наблюдаемый максимум ЕЗ — 4170). Этот детерминированный лексический гроху вдохновлён обучаемой компрессией [24, 25], но не является классификатором LLMingua-2. Явная формула и описание граничного поведения исключают приписывание эффекта модели, которая фактически не запускалась.

## 4.6 Селективный перезапуск

### Содержательный смысл.

*Селективный перезапуск* — механизм экономии токенов и денежного бюджета при неудачной попытке решения. Вместо того чтобы при отказе критика перезапускать всю итерацию с нуля или вообще прекращать работу, процедура отбирает  $M \leq K$  лучших по оценке критика кандидатов, поочерёдно прогоняет их через внешнюю верификацию `ExternalVerify` и, если ни один не прошёл, формирует новый промпт *только* на основе *подтверждённых* инструментами сигналов ошибки (сообщений тестов, трасс исключений, вывода статического анализатора), а не субъективных комментариев критика. Это ограничивает распространение языковых галлюцинаций между итерациями и уменьшает число дорогих повторных запусков. Алгоритм 3 формализует процедуру.

### Расшифровка входов и выходов Алгоритма 3.

$\{y_1, \dots, y_K\} \subset \mathcal{Y}$  — набор кандидатных решений, полученных от роли  $r_{\text{work}}$  на текущей итерации.

$\{s_1, \dots, s_K\} \subset [0, 1]$  — соответствующие оценки от роли-критика  $r_{\text{crit}}$ .

$M \in \{1, \dots, K\}$  — число кандидатов, допускаемых к *дорогой* внешней проверке (в базовой конфигурации  $M = \min(K, 3)$ ).

`ExternalVerify`( $y_m$ )  $\in \{0, 1\}$  — внешняя проверка (вызов тестового раннера, линтера, статического анализатора); возвращает 1 только при полном успехе.

---

**Algorithm 3** Селективный перезапуск по подтверждённым сигналам

---

**Require:** задача  $x$ , кандидаты  $\{y_1, \dots, y_K\}$ , оценки  $\{s_k\}$

**Ensure:**  $(y^*, v^*, M^*, e^*)$

```
1: отсортировать  $\{y_k\}$  по убыванию  $s_k$ ; оставить верхние  $M \leq K$ 
2: for каждый  $y_m$  из отобранных do
3:    $(v_m, M_m, e_m) \leftarrow r_{\text{self}}(y_m)$ 
4:   if  $v_m = 1$  then return  $(y_m, 1, M_m, e_m)$ 
5:   end if
6: end for
7: сформировать один корректирующий промпт из  $x$ , лучшего кандидата и
    $\text{Truncate}(e_m, 600)$ 
8:  $y' \leftarrow \text{LargeModelRerun}(\text{промпт})$ 
9:  $(v', M', e') \leftarrow r_{\text{self}}(y')$ 
10: return  $(y', v', M', e')$ 
```

---

$(y^*, v^*, M^*, e^*)$  — исправленное решение и результат его внешней проверки; общий лимит  $R$  проверяется вызывающим Алгоритмом 1.

*Почему именно подтверждённые сигналы.*

Восстанавливая промпт только из объективных сообщений инструментов, алгоритм обрывает обратную связь «критик подтверждает собственную ошибку  $\rightarrow$  следующее поколение наследует её», которая систематически ухудшает сходимость в чисто языковых Self-Refine/Reflexion-циклах [7, 8].

## 5 Аналитическая модель токенных и стоимостных затрат

### 5.1 Структура аналитических соотношений

Анализ в данном разделе включает два типа соотношений. К первому относятся прямые балансовые равенства и неравенства, задающие структуру токенных и денежных затрат. Ко второму относятся утверждения о сравнении архитектур при фиксированных предположениях относительно длины контекста, стоимости координации и внутренней критики. Такое разделение позволяет отличить общие расчётные соотношения от результатов, требующих дополнительных условий.

### 5.2 Токенные бюджеты Classical-MAS и Hybrid-INoT

Перед формулировкой утверждения о преимуществе по токенам введём обозначения для *всех* слагаемых, из которых складываются бюджеты  $T_{\text{MAS}}$  и  $T_{\text{INoT}}$ , и зафиксируем соответствующие предположения. Все величины далее выражены в токенах модели и относятся к одной задаче.

$T_{\text{MAS}}$  — суммарный токенный расход агента Classical-MAS на одну задачу (случайная величина по распределению  $\mathbb{P}$ ).

$T_{INoT}$  — суммарный токeнный расход агента Hybrid-INoT на одну задачу.  
 $T_{MAS}^{ctx}$  — вклад повторной передачи общего контекста в бюджет Classical-MAS.  
 $T_{coord}^{MAS}$  — вклад координационных сообщений между ролями в бюджет Classical-MAS.  
 $T_{gen}$  — совокупный вклад собственно генерации кандидатов и прочих членов, не зависящих линейно от числа внешних ролей  $|\mathcal{A}|$  (одинаково присутствующий в обеих архитектурах и потому сокращающийся при вычитании).  
 $T_{cmp}$  — стоимость однократной компрессии контекста  $\mathcal{C}_0 \mapsto \hat{\mathcal{C}}$ .  
 $T_{core}^{add}$  — дополнительная стоимость внутренней критики в Hybrid-INoT сверх обычной генерации (расход на ролевые префиксы и развёрнутый внутренний дебат).  
 $T_{check}$  — суммарная стоимость внешних верификаций (запусков тестов/линтера, передачи их вывода обратно в модель) за одну задачу.  
 $T_{verin}$  — случайная величина стоимости повторных запусков при провале внешней верификации.  
 $\alpha \in (0, 1]$  — доля ролей Classical-MAS, читающих полный контекст (см. Предположение 1).  
 $\gamma \geq 0$  — коэффициент, ограничивающий стоимость компрессии:  $T_{cmp} \leq \gamma|\mathcal{C}_0|$ .  
 $\beta \geq 0$  — коэффициент, ограничивающий дополнительную стоимость критики:  $T_{core}^{add} \leq \beta|\hat{\mathcal{C}}|$ .  
 $I \in \mathbb{N}$  — верхний предел числа итераций в обеих архитектурах.  
 $\bar{h}$  — средняя длина координационного сообщения между ролями (в токенах).

Неотрицательность  $\beta$  и  $\gamma$  сама по себе не гарантирует преимуществ одной архитектуры. Точное достаточное условие  $A_0 > 0$  ниже совместно зависит от  $I, \alpha|\mathcal{A}|, q_{\max}, \beta$  и  $\gamma$ ; его предпосылки подлежат эмпирической проверке для конкретной реализации.

**Предположение 1.** На каждой из  $I$  итераций Classical-MAS не менее  $\alpha|\mathcal{A}|$  ролей,  $\alpha \in (0, 1]$ , читают общий контекст  $\mathcal{C}_0$  целиком. Hybrid-INoT использует выбранный контекст  $\hat{\mathcal{C}}$ , для которого  $|\hat{\mathcal{C}}| \leq q_{\max}|\mathcal{C}_0|$  при фиксированном  $q_{\max} \leq 1$ .

**Предположение 2.** Стоимость однократной компрессии ограничена как  $T_{cmp} \leq \gamma|\mathcal{C}_0|$ , дополнительная стоимость внутренних ролей — как  $T_{core}^{add} \leq \beta|\hat{\mathcal{C}}|$ , а модуль разности всех остальных слагаемых генерации, верификации и повторных запусков двух систем не превышает константы  $K_0$ , не зависящей от  $|\mathcal{C}_0|$ .

**Утверждение 2** (Достаточное балансовое условие). Пусть

$$A_0 = I\alpha|\mathcal{A}| - (I + \beta)q_{\max} - \gamma.$$

При Предположениях 1–2, если  $A_0 > 0$ , то

$$\mathbb{E}[T_{MAS} - T_{INoT}] \geq A_0|\mathcal{C}_0| - K_0, \quad (7)$$



и ожидаемая разность токенов расходов положительна при  $|C_0| > K_0/A_0$ .

*Балансовый вывод* Из Предположения 1

$$T_{\text{ctx}}^{\text{MAS}} \geq I\alpha|\mathcal{A}| \cdot |C_0|.$$

Для Hybrid-INoT сумма повторной передачи выбранного контекста, дополнительного расхода внутренних ролей и компрессии ограничена:

$$(I + \beta)|\hat{C}| + \gamma|C_0| \leq ((I + \beta)q_{\max} + \gamma)|C_0|.$$

После вычитания и применения границы  $K_0$  ко всем остаточным различиям получаем (7). При  $A_0 > 0$  её правая часть положительна ровно при  $|C_0| > K_0/A_0$ .  $\square$

**Область применимости утверждения.**

Утверждение 2 — достаточный балансовый результат для полного системного пакета, а не причинная теорема об одной оркестрации и не валидация маршрутизатора. Неотрицательные координационные расходы Classical-MAS могут усилить границу, но не используются как недоказанная асимптотическая нижняя оценка. ЕЗ оценивает склоны пакетов В2/В3; поскольку пересечение не заключено внутри сетки, эксперимент не идентифицирует ни  $K_0/A_0$ , ни deployment-порог.

### 5.3 Тожество числа запросов и граница выходных токенов

Реализация получает все  $K$  кандидатов, план и критику одним основным вызовом на итерацию; поэтому умножение одного prompt на  $K$  дважды учитывало бы общий контекст. Пусть  $N_{\text{core}} \leq I$  и  $N_{\text{rerun}} \leq \min(I, R)$  — фактическое число основных и повторных вызовов, а  $P$  и  $O$  — записанные провайдером входные и выходные токены одного вызова. Для внутреннего маршрута выполняется тождество

$$T_{\text{INoT}}^{\text{internal}} = \sum_{i=1}^{N_{\text{core}}} (P_{\text{core},i} + O_{\text{core},i}) + \sum_{j=1}^{N_{\text{rerun}}} (P_{\text{rerun},j} + O_{\text{rerun},j}). \quad (8)$$

В зафиксированной реализации серверные лимиты выхода равны  $O_{\text{core},i} \leq 2048 + 700K$  и  $O_{\text{rerun},j} \leq 1024$ , откуда

$$T_{\text{out}} \leq N_{\text{core}}(2048 + 700K) + 1024N_{\text{rerun}}.$$

Компрессия детерминирована и не вызывает модель. Бюджет суммы блоков равен 4096 проху-токенам, но разделители не входят в этот счётчик, поэтому сериализованный контекст ЕЗ достигает 4170 проху-токенов. Жёсткой границы полного входа нет: постановка задачи не ограничена, а проху-счётчик может отличаться от токенизации провайдера. Код применяет предзапросный денежный сар в общем API-клиенте, но не per-task token counter. Поэтому мы сообщаем тождество записанного расхода и output-only границы, но не заявляем жёсткую гарантию полного токенового бюджета.

## 5.4 Условие экономической эффективности малой модели самопроверки

### Обозначения для формулы стоимости инференса.

Пусть агент в процессе решения одной задачи делает несколько вызовов крупной (large, верхний индекс  $L$ ) и малой (small, верхний индекс  $S$ ) моделей с индексом  $i$ , пробегающим по вызовам. Для каждого вызова обозначим:

$p_i^L, r_i^L$  — число входных (prompt) и выходных (response) токенов  $i$ -го вызова крупной модели.

$p_i^S, r_i^S$  — то же для  $i$ -го вызова малой модели.

$c_L^{\text{in}}, c_L^{\text{out}}$  — цены *крупной* модели за один входной и выходной токен соответственно (в валюте расчёта, обычно долл. США за токен; у провайдеров API обычно указывается цена за  $10^6$  токенов, и значения пересчитываются делением на  $10^6$ ).

$c_S^{\text{in}}, c_S^{\text{out}}$  — то же для малой модели.

$$C_{\text{inf}} = c_L^{\text{in}} \sum_i p_i^L + c_L^{\text{out}} \sum_i r_i^L + c_S^{\text{in}} \sum_i p_i^S + c_S^{\text{out}} \sum_i r_i^S. \quad (9)$$

Формула (9) — прямая балансовая запись: денежная стоимость инференса на одну задачу равна сумме по всем вызовам произведений числа токенов на соответствующий тариф, отдельно для входных/выходных токенов и крупной/малой моделей.

### Обозначения для условия эффективности.

Рассмотрим схему, в которой после получения кандидата  $y$  крупной моделью он предварительно проверяется *малой* моделью; если малая модель сигнализирует об ошибке, запускается повторная генерация крупной моделью. Введём:

$C_{\text{check}}^S$  — денежная стоимость одной предварительной проверки малой моделью (включает входной и выходной токены расход проверочного вызова, пересчитанный в деньги по (9)).

$C_{\text{rerun}}^L$  — денежная стоимость одного повторного запуска крупной модели (при провале текущего кандидата).

$\rho_0$  — вероятность того, что потребуется дорогостоящий повторный запуск *в базовой схеме без малой модели самопроверки* (то есть доля ошибочных кандидатов, не выловленных ни одним дешёвым механизмом).

$\rho_1$  — та же вероятность *при наличии* предварительной проверки малой моделью; по построению  $0 \leq \rho_1 \leq \rho_0 \leq 1$ , причём часть ошибочных кандидатов малая модель отсекает, прежде чем они попадут в крупную.

### Вывод условия.

Сравним ожидаемые расходы на одну задачу в двух сценариях.

*Сценарий 0* (без малой модели): расход = 0 при успехе; с вероятностью  $\rho_0$  возникает повторный запуск крупной модели стоимостью  $C_{\text{rerun}}^L$ . Ожидаемые дополнительные расходы:  $\mathbb{E}_0 = \rho_0 \cdot C_{\text{rerun}}^L$ .

*Сценарий 1* (с малой моделью): *всегда* платим  $C_{\text{check}}^S$  за предварительную проверку; дополнительно с вероятностью  $\rho_1$  всё ещё нужен повторный запуск стоимостью  $C_{\text{rerun}}^L$ . Ожидаемые дополнительные расходы:  $\mathbb{E}_1 = C_{\text{check}}^S + \rho_1 \cdot C_{\text{rerun}}^L$ .

В этой двухсценарной модели с ровно одной предварительной проверкой, не более чем одним предотвращённым перезапуском и постоянной стоимостью перезапуска малая модель экономически эффективна тогда и только тогда, когда  $\mathbb{E}_1 < \mathbb{E}_0$ , то есть

$$C_{\text{check}}^S + \rho_1 \cdot C_{\text{rerun}}^L < \rho_0 \cdot C_{\text{rerun}}^L, \quad (10)$$

что после переноса  $\rho_1 \cdot C_{\text{rerun}}^L$  в правую часть даёт

$$C_{\text{check}}^S < (\rho_0 - \rho_1) \cdot C_{\text{rerun}}^L. \quad (11)$$

**Содержательная расшифровка (11).** Левая часть — фиксированная стоимость предварительной проверки. Правая часть — ожидаемая экономия *при условии, что интегрированная политика действительно предотвращает часть запусков крупной модели*. Поэтому неравенство является перспективным условием проектирования. Сохранённый E4 содержит только агрегированное согласие screening-процедуры и не реализует и не идентифицирует эффект политики  $\rho_0 - \rho_1$ .

## 6 Система метрик

### 6.1 Основные метрики и обоснование формы агрегации

Основными количественными характеристиками работы агента в настоящем исследовании выступают токенная эффективность  $U_{\text{tok}}$ , денежная эффективность  $Q_{\$}$  и доля полностью проверенных решений VCR (Verified Candidate Rate). Приведём сначала определения, затем — расшифровку всех входящих величин.

$$U_{\text{tok}} = \frac{Q}{T}, \quad (12)$$

$$Q_{\$} = \frac{Q}{C_{\text{inf}}}, \quad (13)$$

$$\text{VCR} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{tests}_i = 1 \wedge \text{static}_i = 1 \wedge \text{sec}_i = 1]. \quad (14)$$

**Расшифровка (12)–(14).**

$Q$  — агрегированная скалярная оценка качества решения задачи, принимающая значения на  $[0, 1]$ . По построению  $Q$  совпадает с ожидаемым значением функции успеха  $\mathbb{E}[\mathcal{V}(Y)]$  (см. (2)), а в случае неоднородного датасета — с взвешенной суммой по классам задач  $Q = \sum_b w_b Q_b$ , где  $Q_b \in [0, 1]$  — средняя успешность по классу  $b$  (например, «короткие задачи HumanEval», «длинный контекст SWE-bench»), а  $w_b \geq 0$  — доля класса в оцениваемой выборке при  $\sum_b w_b = 1$ .

$T$  — средний токeнный расход на одну задачу (в токeнах);  $U_{\text{tok}} = Q/T$  — «качество, получаемое с 1000 токeнов», то есть показатель токeнной эффективности.  $C_{\text{inf}}$  — средняя денежная стоимость инференса на одну задачу (см. (9));  $Q_{\$} = Q/C_{\text{inf}}$  — «качество, получаемое на 1 доллар», то есть показатель денежной эффективности.

$N$  — число задач в оцениваемой выборке.

$\text{tests}_i \in \{0, 1\}$  — результат прогона автоматического тестового набора по  $i$ -й задаче: 1, если все тесты пройдены, 0 иначе.

$\text{static}_i \in \{0, 1\}$  — результат статического анализа (линтера/типизатора): 1, если не сообщено ни одной ошибки из набора критических правил, 0 иначе.

$\text{sec}_i \in \{0, 1\}$  — результат проверки безопасности (например, Bandit / Semgrep для Python, CodeQL для общих целей): 1, если не обнаружено ни одной уязвимости, 0 иначе.

$\mathbb{I}[\cdot]$  — индикаторная функция: равна 1, если указанное в скобках условие истинно, и 0 иначе. В частности,  $\mathbb{I}[\text{static}_i = 1]$  принимает значение 1, если статический анализатор для  $i$ -й задачи не выявил критических замечаний, и 0 в противном случае.

$\wedge$  — логическая конъюнкция («и»): выражение  $\text{tests}_i = 1 \wedge \text{static}_i = 1 \wedge \text{sec}_i = 1$  истинно лишь тогда, когда *одновременно* выполнены все три условия.

**Содержательный смысл VCR.** Формула (14) считает *долю* задач, на которых выход агента одновременно прошёл все три класса объективных проверок — функциональные тесты, статический анализ и проверку безопасности. Решение засчитывается только при успехе по всем трём критериям: индикатор в сумме даёт 1, если и только если все три условия истинны, и 0 иначе. Множитель  $\frac{1}{N}$  нормирует сумму на число задач  $N$ , переводя её в долю от 0 до 1 (в процентах — от 0% до 100%). Таким образом, запись  $\frac{1}{N} \sum_i \mathbb{I}[\dots]$  — это обычная *средняя доля успехов*, а не произвольный коэффициент  $1/3$ : тройки условий здесь нет, есть  $N$  задач и усреднение по ним.

### Почему именно три критерия.

Три компонента индикатора — тесты, статический анализ и проверка безопасности — задают *минимально достаточный* набор объективных сигналов для патча в инженерной постановке. Функциональные тесты фиксируют корректность, статический анализ — отсутствие явных ошибок и нарушений стиля, проверка безопасности — отсутствие уязвимостей. Конъюнкция консервативна: частичный успех (например, «тесты пройдены, но есть уязвимость») не засчитывается, что соответствует практикам промышленного CI/CD.

## 6.2 Агрегированная метрика успешности и её форма

Для агрегированной оценки успешности используется форма

$$\text{MAS}_i = \min(\text{Success}_i, \text{Verified}_i) \cdot M_i^\lambda, \quad \lambda \in [0, 1], \quad \lambda = 0.5. \quad (15)$$

Обоснование этой конструкции состоит в следующем. Во-первых, величины Success и Verified являются необходимыми условиями засчитывания решения: patch, не прошедший внешнюю проверку, не должен получать высокий итоговый балл. Во-вторых, множитель  $M_i^\lambda$  моделирует влияние поддерживаемости на итоговую оценку; параметр  $\lambda$  регулирует силу этого влияния. В базовой конфигурации используется  $\lambda = 0.5$ ; анализ E2d обнаружил семь инверсий ранжирования при изменении  $\lambda$  и весов, поэтому составная метрика рассматривается только как вспомогательная.

### 6.3 Веса поддерживаемости и их обоснование

$$M_i = w_{cc}(1 - \widehat{CC}_i) + w_{dup}(1 - \widehat{Dup}_i) + w_{warn}(1 - \widehat{Warn}_i) + w_{doc}\widehat{Doc}_i. \quad (16)$$

**Таблица 2** Веса поддерживаемости и их связь с подкатегориями ISO/IEC 25010:2023 [30]

| Параметр   | Значение | Обоснование                                                                                               |
|------------|----------|-----------------------------------------------------------------------------------------------------------|
| $w_{cc}$   | 0.35     | ISO 25010 Modifiability; цикломатическая сложность [31] — наиболее прямой прокси стоимости сопровождения. |
| $w_{dup}$  | 0.25     | ISO 25010 Reusability; дублирование — ключевой сигнал технического долга.                                 |
| $w_{warn}$ | 0.20     | ISO 25010 Analysability; warnings — поверхностный, но надёжный сигнал.                                    |
| $w_{doc}$  | 0.20     | ISO 25010 Modifiability; покрытие документацией как прокси понятности.                                    |

### Анализ чувствительности

В E2d проверяется устойчивость ранжирования конфигураций B0–B3 при вариации  $(w_{cc}, w_{dup}, w_{warn}, w_{doc})$  в  $\pm 20\%$  и при  $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$  в (15). Инверсия ранжирования при таких вариациях рассматривается как признак чувствительности метрики к выбранной параметризации.

### 6.4 Метрики надёжности

$$SCR = \frac{\#\{\text{ошибочные кандидаты, выявленные процедурой самопроверки}\}}{\#\{\text{все ошибочные кандидаты}\}}, \quad (17)$$

$$RFR = \frac{\#\{\text{патчи без отката}\}}{\#\{\text{применённые патчи}\}}. \quad (18)$$

## 7 Эмпирическая валидация

### 7.1 Воспроизводимость, исходные данные и статистический протокол

Исследовательский стенд опубликован в репозитории INoT\_Research [34]; все результаты привязаны к commit 010211ee5775185e8330ab6cde06e912c2adcb9e.

Проверка `python -m inot check` завершилась без ошибок, а `pytest` дал 15 успешных тестов. Исходные JSON находятся в `results/slide_e2e3_real`, `results/e6` и `reproducibility/results/20260726_flash_lite`; переносимые относительные пути, SHA-256, агрегаты и код рисунков сохранены в `reproducibility/evidence_snapshot.json`. Проверка сетки нашла 60/60 ожидаемых строк Pro и 240/240 строк Flash-Lite без дубликатов ключей, пропущенных ячеек, расхождений токеного баланса и `usage trace`. Единица анализа — задача  $\times$  архитектура  $\times$  целевой контекст  $\times$  seed.

Основной набор данных — канонический HumanEval [22]. Pro-пилот использует задачи HumanEval/0–4, а повторный Flash-Lite-прогон — HumanEval/0–19 в исходном порядке. Исторический контекст строится детерминированно (seed 42) из постановок и эталонных решений *других* задач HumanEval; эталон целевой задачи не добавляется. Блоки присоединяются целиком, поэтому фактическая длина может превышать целевую. Для  $n = 20$  при целевых  $\{256, 1024, 4096, 16384\}$  фактические средние длины равны соответственно  $\{371.2, 1154.8, 4210.6, 16524.3\}$  токена, а диапазоны —  $\{259\text{--}545, 1024\text{--}1411, 4100\text{--}4415, 16389\text{--}16946\}$ . Такой контекст воспроизводим, но синтетичен и не моделирует историю реального репозитория во всей полноте.

Сравниваются B2 Classical-MAS (planner, worker и critic как отдельные API-вызовы над несжатым контекстом) и B3 Hybrid-INoT (объединённые ролевые инструкции и детерминированный отбор контекста свыше 4096 токенов). После генерации выполняется внешняя проверка. Следовательно, основной estimand — end-to-end контраст пакетов, а не изолированный эффект оркестрации. E3 всегда запускает B3 во внутреннем режиме и не проверяет RouteMode. Про-пилот выполнен через `google/gemini-3.1-pro-preview`, репликация — через `google/gemini-3.1-flash-lite`. Цены \$2/\$12 и \$0.25/\$1.50 за миллион входных/выходных токенов зафиксированы 26 июля 2026 г. [35, 36]. API-псевдонимы не являются неизменными снимками весов.

Для каждой длины контекста образуется пара B2–B3 на одной задаче и одном seed. Используются

$$S_T = 100 \left( 1 - \frac{\bar{T}_{B3}}{\bar{T}_{B2}} \right), \quad S_C = 100 \left( 1 - \frac{\bar{C}_{B3}}{\bar{C}_{B2}} \right), \quad (19)$$

$$\Delta U_{\text{tok}} = 100 \left( \frac{U_{\text{tok}}^{B3}}{U_{\text{tok}}^{B2}} - 1 \right). \quad (20)$$

Средние, минимумы и число пар выше порога рассматриваются как описательные estimand конечного набора. Критерий Вилкоксона [32] сообщается только как условный анализ симметричного сдвига распределения и не тестирует приведённое среднее. Для 15%-го подкритерия односторонний signed-rank тест применяется к  $\Delta U_{\text{tok}} - 15\%$ , а семья четырёх контекстов корректируется процедурой Холма [33]. Диапазоны по  $10^4$  task-level ресэмплированиям показывают устойчивость к перевзвешиванию наблюдаемых задач, но не популяционное покрытие. Для качества рассчитывается вредная дискордантность; нулевая Clopper–Pearson граница служит iid-биномиальным планировочным ориентиром, а не design-valid

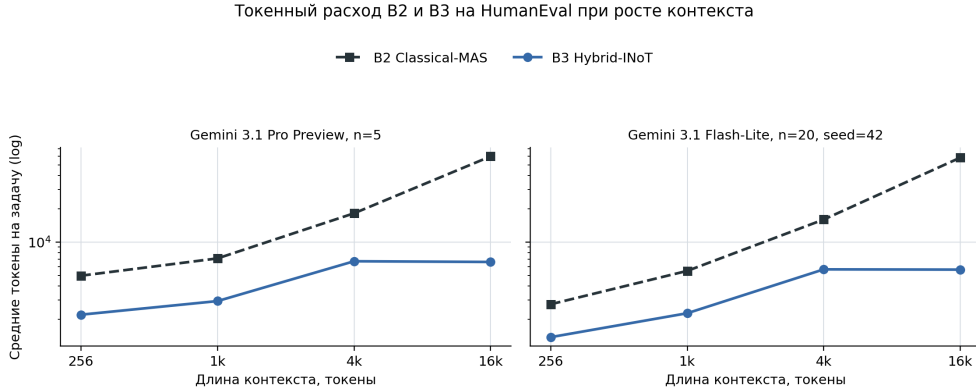
выводом для первых неслучайных задач HumanEval. Четыре повторных контекста не считаются независимыми исходами, а один сохранённый API-прогон не оценивает вариативность seed или повторных вызовов провайдера.

## 7.2 E3 — Масштабирование по длине контекста

Таблица 3 содержит непосредственно наблюдаемые средние. В обоих прогонах расход B2 растёт почти линейно с добавляемым контекстом, тогда как B3 после 4096 целевых токенов остаётся около 5.6–6.7 тыс. токенов вследствие компрессии.

**Таблица 3** Реальные API-измерения B2 и B3 на HumanEval

| Модель                  | Контекст | $T_{B2}$ | $T_{B3}$ | $S_T$ , % | $S_C$ , % | pass@1 B2/B3 |
|-------------------------|----------|----------|----------|-----------|-----------|--------------|
| Pro ( $n = 5$ )         | 256      | 4964     | 2203     | 55.6      | 43.1      | 1.00/1.00    |
|                         | 1024     | 7117     | 2925     | 58.9      | 46.8      | 1.00/1.00    |
|                         | 4096     | 18218    | 6706     | 63.2      | 49.1      | 1.00/1.00    |
|                         | 16384    | 59631    | 6608     | 88.9      | 79.0      | 1.00/1.00    |
| Flash-Lite ( $n = 20$ ) | 256      | 2738     | 1379     | 49.6      | 26.0      | 1.00/1.00    |
|                         | 1024     | 5487     | 2271     | 58.6      | 41.8      | 1.00/1.00    |
|                         | 4096     | 16007    | 5664     | 64.6      | 56.9      | 1.00/1.00    |
|                         | 16384    | 58110    | 5633     | 90.3      | 86.9      | 1.00/1.00    |



**Рис. 1** Средний суммарный токенный расход на задачу. Штриховая линия с квадратами — B2 Classical-MAS; сплошная линия с кругами — B3 Hybrid-INoT. Ось  $y$  логарифмическая. Слева показан сохранённый Pro-пилот ( $n = 5$  на точку), справа — выполненная для статьи репликация Flash-Lite ( $n = 20$ , seed 42).

Покажем вычисление для Pro при 16 384 целевых токенах:

$$S_T = 100 \left( 1 - \frac{6608.4}{59631.4} \right) = 88.9\%, \quad S_C = 100 \left( 1 - \frac{0.0294908}{0.1407228} \right) = 79.0\%.$$

Поскольку  $\text{pass}@1$  обеих конфигураций равен 1, агрегированное отношение средних даёт

$$U_{B2} = \frac{1}{59631.4} = 1.677 \cdot 10^{-5}, \quad U_{B3} = \frac{1}{6608.4} = 1.513 \cdot 10^{-4},$$

то есть агрегированное отношение средних даёт рост  $U_{\text{ток}}$  802.4%. Основной парный estimand сначала вычисляет относительное отношение внутри каждой задачи, а затем усредняет: 805.6%, task-resampling диапазон устойчивости 756.0–850.4%.

В Pro-пилоте все парные выигрыши превышают 15%, но при пяти однонаправленных парах условное одностороннее signed-rank  $p = 0.03125$  в каждой точке не проходит поправку Холма. В Flash-Lite-прогоне средние по конечному набору внутрипарные приросты равны 106.5% (task-resampling диапазон 87.5–126.1), 144.8% (131.9–158.3), 183.3% (177.3–189.6) и 934.6% (910.8–959.6). Все 20 пар превышают 15% в каждой ячейке; условное одностороннее signed-rank  $p = 9.54 \cdot 10^{-7}$  проходит поправку Холма. Это вывод о наблюдаемой выборке, а не о среднем популяции.

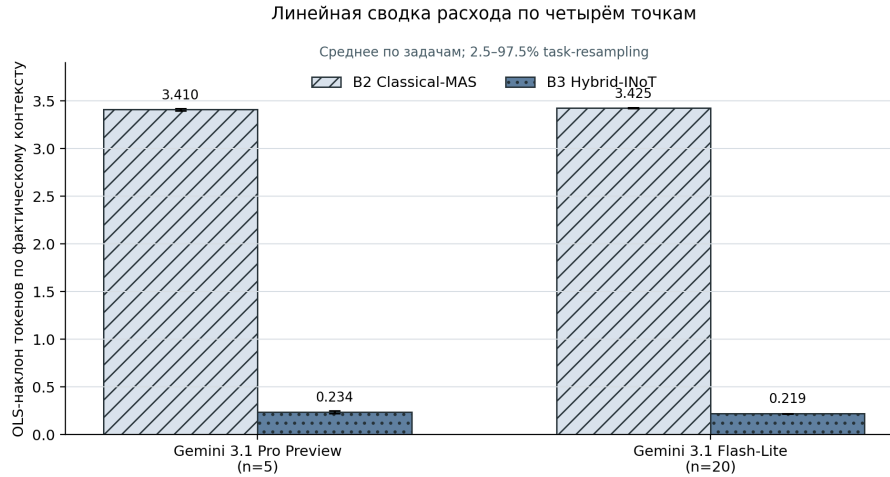
Для каждой задачи дополнительно оценена четырёхточечная OLS-сводка суммарных токенов по *фактической* длине контекста. Средние значения Flash-Lite равны 3.425 (task-resampling диапазон 3.421–3.430) для B2 и 0.219 (0.216–0.222) для B3 токена на токен контекста; все парные разности положительны (средняя 3.206, диапазон 3.201–3.212; условное signed-rank  $p = 1.91 \cdot 10^{-6}$ ). Для Pro получено 3.410 против 0.234 при условном  $p = 0.0625$  (рис. 2). Поскольку B3 пересекает границу включения компрессии 4096 токенов, коэффициенты описывают всю сетку и не являются локальными производными; диапазоны также не включают неопределённость seed и повторных API-прогонов.

$\text{Pass}@1 = 1.0$  для обеих архитектур создаёт потолочный эффект. Среди 20 уникальных Flash-Lite-пар вредных исходов нет. В iid-биномиальной модели односторонняя 95%-я нулевая граница-ориентир равна 13.91 п.п.; чтобы опустить её ниже 2 п.п., требуется не менее 149 пар без вреда, а для всех 164 задач граница составила бы 1.81 п.п. Поскольку взяты первые 20, а не случайная выборка, номинальные уровни служат расчётом мощности, но не гарантией популяционного покрытия. Следовательно, ресурсный подкритерий описательно выполнен в фиксированной выборке, но полная H1 не доказана. E3 не заключает пересечение внутрь сетки: поведение ниже 256 целевых токенов неизвестно, поэтому ни теоретический  $\tau^*$ , ни порог маршрутизации не идентифицированы.

### 7.3 E2 — Абляции, критик и параллелизм

Абляционный Pro-пилот содержит пять задач.  $\text{pass}@1$  равен 1.0 для полной B3 и вариантов без компрессии, критика и самопроверки. Попарные разности агрегированной успешности для B3 против этих трёх вариантов равны  $-0.0037$ ,  $-0.0262$  и  $0.0000$ , с  $p = 1.0$ ,  $0.5$  и  $1.0$ ; ни одна разность не значима после поправки Холма. Доли токенового расхода компонентов составляют 36.6% для компрессии, 29.7% для критика и 33.8% для самопроверки, но это *накладные расходы*, а не доказанный независимый вклад в качество. На подмножестве ошибок B0 оказалось





**Рис. 2** Task-level четырёхточечные OLS-сводки суммарных токенов по фактической длине контекста. Столбцы — средние по задачам, интервалы — 2.5–97.5% task-resampling диапазоны устойчивости. Estimand относится к полным пакетам B2/B3 и не является локальной производной через границу компрессии.

только две задачи; наблюдаемая доля ошибочных одобрений критика равна нулю, что недостаточно для вывода о надёжности критика.

В E2c среднее время B2 равно 19.28 с, B3 — 9.76 с; относительное изменение B3 составляет  $-49.4\%$ , task-resampling диапазон разности B3–B2 равен  $[-13.05; -6.83]$  с, а условный критерий Вилкоксона при  $n = 5$  даёт  $p = 0.0625$ . Отдельный детерминированный микротест инструментов действительно показывает, что последовательное выполнение медленнее параллельного на  $148.8\%$ , однако прямое сравнение архитектур не показало ожидаемого проигрыша B3. Поэтому H3 в данном пилоте *не поддерживается*; микротест подтверждает лишь механизм, который предстоит изолировать на задачах с реальными независимыми инструментами. В E2d получено семь инверсий ранжирования при вариации весов и  $\lambda$ ; канонически первым оказался вариант без критика. Это указывает на неустойчивость составной метрики и исключает сильный вывод о необходимости каждого компонента.

## 7.4 E4 — Разведочная диагностика согласия малой модели

В 20 случаях HumanEval и 20 случаях приближённого SWE-bench Lite Gemini 3.1 Pro Preview генерировал решение, а Flash-Lite выдавал PASS/REJECT. В сохранённом файле есть агрегированные счётчики, но нет решений малой модели по каждому случаю, текстов кандидатов, индивидуальных стоимостей и контрфактических исходов перезапуска. Отклонение неудачного кандидата в этом harness само по себе не предотвращает реализованный повторный вызов. Поэтому исходный policy-estimand из (11) и H2 невоспроизводим и причинно не оценён; можно сообщить только согласие с сохранённым исходом внешней проверки.

**Таблица 4** Агрегированная диагностика screening-процедуры

| Набор           | $n$ | Верно/pass | Верно/reject | Ошибка/pass | Ошибка/reject |
|-----------------|-----|------------|--------------|-------------|---------------|
| HumanEval       | 20  | 13         | 0            | 0           | 7             |
| SWE-bench Lite* | 20  | 3          | 2            | 12          | 3             |

Для HumanEval согласие равно 1.00 (95%-й Clopper–Pearson reference-интервал 0.832–1.000), sensitivity, specificity и balanced accuracy также равны 1.00; средняя цена проверки — 0.555% стоимости генерации. Для приближённого SWE-bench согласие 0.30 (reference-интервал 0.119–0.543), sensitivity 0.60, specificity 0.20, balanced accuracy 0.40; цена проверки — 0.846% генерации. Номинальные интервалы предполагают iid-случаи и служат лишь описательными ориентирами фиксированных наборов. Звёздочка существенна: загрузчик не запускает официальный контейнерный SWE-bench harness [20], а использует извлечение кода и заглушку `check(_)`. Эти агрегаты мотивируют будущий интегрированный policy-эксперимент, но не поддерживают H2.

## 7.5 E6 — Межмодельная чувствительность

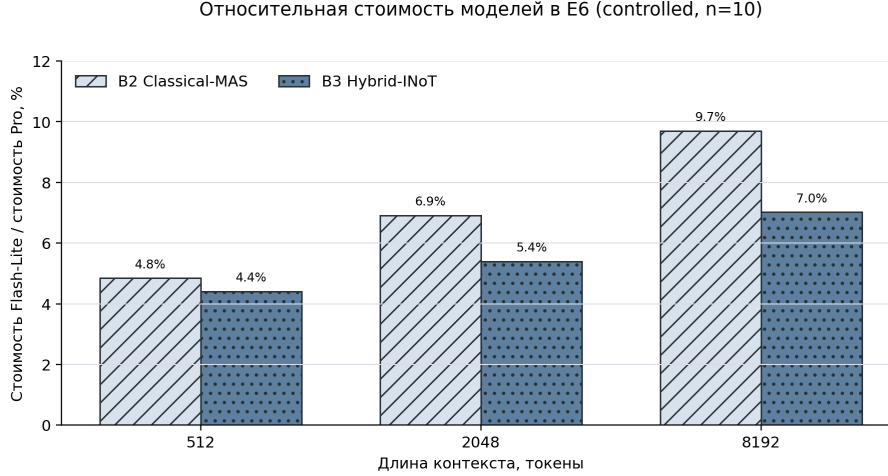
Сохранённый E6 сравнивает Pro и Flash-Lite на контролируемом наборе из десяти случаев для каждой пары архитектура–контекст. Набор содержит повторения трёх семейств функций (`reverse_words`, `count_peaks`, `normalize_path`); `pass@1` равен 1.0 во всех ячейках, поэтому эксперимент измеряет стоимость на простом потолочном наборе. При длинах 512, 2048 и 8192 стоимость Flash-Lite составляет 4.8, 6.9 и 9.7% стоимости Pro для B2 и 4.4, 5.4 и 7.0% для B3 (рис. 3). Парные различия  $U_{\text{tok}}$  значимы ( $p = 0.001953$ ; все сравнения проходят поправку Холма), но из-за потолка качества этот результат нельзя обобщать на сложные задачи.

## 7.6 E5 — Экстраполяция стоимости API-инференса

Полный ТСО-эксперимент не был завершён: дополнительный запуск был остановлен лимитом провайдера до сохранения `runs.json` и исключён из анализа. Вместо подмены отсутствующих данных выполнена явно ограниченная аналитическая экстраполяция из двадцати парных Pro-случаев E3 (пять задач на каждую из четырёх длин, равные веса). Для  $N$  задач

$$\Delta C_{\text{inf}}(N) = N \cdot \frac{1}{20} \sum_{i=1}^{20} (C_{B2,i} - C_{B3,i}). \quad (21)$$

Средняя разность равна \$0.042996 на задачу; при  $N = 10\,000$  это \$429.96 экономии B3. Поскольку одни и те же пять задач повторяются на четырёх длинах контекста, ресэмплирование пяти *кластеров задач* с сохранением всех четырёх ячеек даёт диапазон устойчивости \$417.33–\$442.90, не имеющий design-based популяционного покрытия. Leave-one-task-out оценки лежат в \$425.20–\$435.51. Среднее сокращение входа и выхода составляет 17 146.8 и 725.2 токена на случай; все 20



**Рис. 3** Отношение средней стоимости Gemini 3.1 Flash-Lite к стоимости Gemini 3.1 Pro Preview в контролируемом E6. Штриховки обеспечивают различимость при чёрно-белой печати.

парных разностей положительны. Независимое изменение множителей входной и выходной цены от 0.5 до 1.5 сохраняет положительную экономию во всей сетке (\$214.98–\$644.94; рис. 4).

## 8 Экономическая модель

### 8.1 Граница между инференсом и TCO

$$TCO_{\text{month}} = C_{\text{inf}} + C_{\text{hw}} + C_{\text{energy}} + C_{\text{storage}} + C_{\text{ops}}. \quad (22)$$

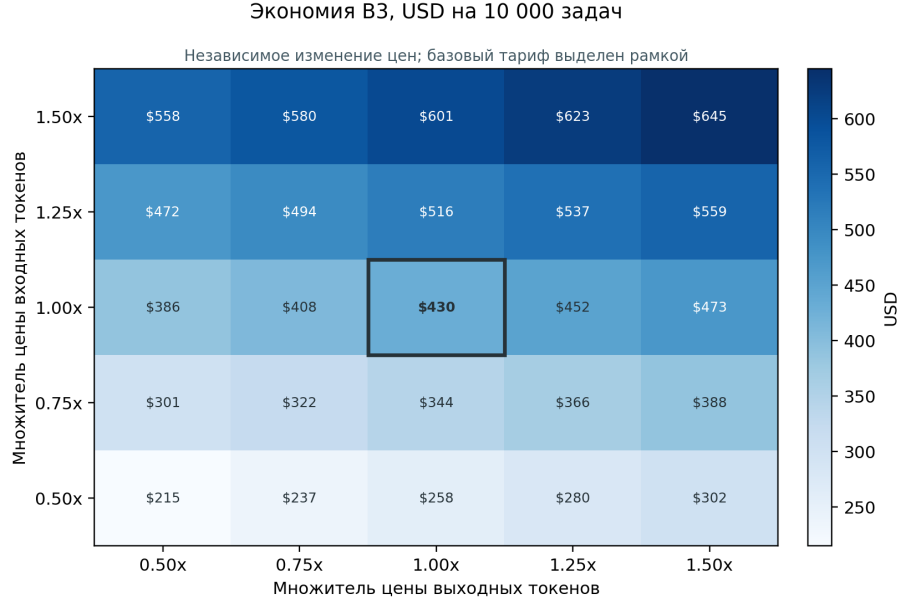
В работе измерено только  $C_{\text{inf}}$  для внешнего API. Аппаратные ресурсы, энергия, хранение, сопровождение, разработка и возможная стоимость задержки не измерялись; следовательно, положительная нижняя граница в E5 доказывает экономию *API-инференса в пилотной смеси*, но не  $\Delta TCO < 0$ . При использовании управляемого API часть остальных слагаемых включена в цену провайдера непрозрачно, что также не позволяет декомпозировать TCO.

### 8.2 Сводный вердикт по гипотезам

## 9 Ограничения и угрозы валидности

### 9.1 Границы интерпретации

1. **Малые и неслучайные выборки.** Использованы первые 5 или 20 задач HumanEval, а не случайная стратифицированная выборка из 164 задач; новый прогон имеет один seed. Это ограничивает статистическую мощность и внешнюю валидность.



**Рис. 4** Экономия API-инференса на 10 000 задач при независимых множителях входной и выходной цены. Контуром отмечен зафиксированный тариф Pго. Это детерминированная ценовая чувствительность, а не популяционная неопределённость.

**Таблица 5** Соответствие исходных гипотез фактическим данным

| Гипотеза         | Вердикт             | Основание                                                                                                                  |
|------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------|
| H1               | в целом не доказана | Каждая Flash-Lite-пара фиксированной выборки превысила ресурсный порог 15%, но неухудшение качества на 2 п.п. не доказано. |
| H2               | не оценена          | E4 сохранил агрегаты screening, но не интегрированную политику перезапуска и не контрфактические стоимости по задачам.     |
| H3               | не поддержана       | Инструментальный микротест показывает пользу параллелизма, но B3 в прямом пилоте быстрее B2; $n = 5$ .                     |
| $\tau^*$ /router | не идентифицированы | Пересечение не заключено в сетке, поведение ниже неё неизвестно, E3 принудительно использует internal B3.                  |

- Синтетический контекст.** Длинный вход составлен из других задач HumanEval, а не из истории репозитория. Он изолирует цену повторной передачи, но не отражает зависимости и шум промышленной кодовой базы.
- Потолок качества.** В B2/B3 и контролируемом E6 pass@1 равен 1.0. Данные показывают различие расхода, но не дают строгого теста неухудшения качества и не исключают иной результат на сложных задачах.
- Пакетный treatment.** B3 меняет топологию вызовов и, свыше 4096 токенов, отбор контекста. Для атрибуции компонентов нужен факторный дизайн; текущий estimand относится к end-to-end пакету.

5. **Неполный SWE-bench.** Используемая проверка не является официальным контейнерным протоколом; вывод о решении issue или переносе на реальные репозитории не делается.
6. **Маршрутизатор не проверен.** E3 всегда вызывает internal B3; точность флагов и контекстный gate не тестировались.
7. **Post-hoc робастность.** Четырёхточечные OLS-сводки, iid-reference граница вреда и task-cluster диапазон ресэмплирования добавлены после аудита пилота. Это прозрачные диагностики, а не пререгистрированные confirmatory endpoints.
8. **Аудитируемость E4.** Per-case решения малой модели и контрфактические исходы перезапуска не сохранены, поэтому H2 остаётся неоценённой.
9. **Дрейф API.** В результатах сохранены имена моделей, параметры, токены, задержка и стоимость, но API-псевдонимы не гарантируют неизменность весов. Неудавшиеся из-за лимита провайдера запуски не включены.
10. **Ограниченная аудитируемость ответа.** `runs.json` хранит метрики и usage, но не полный финальный текст решения; независимая постфактум-проверка каждого ответа ограничена.
11. **Стоимость не равна ТСО.** Рисунок 4 отражает только API-инференс при заданной смеси контекстов. Остальные слагаемые (22) и производственная нагрузка не измерялись.
12. **Архитектурные границы.** Истинно независимые подзадачи, необходимость процессной изоляции, ошибки критика и потеря фактов при компрессии могут устранить преимущество B3. Промежуточные ролевые тексты являются вычислительным механизмом, а не достоверным объяснением поведения модели [13].

## 10 Направления дальнейшего исследования

Следующий этап должен устранять выявленные источники неопределённости, а не повторять уже выполненный пилот:

1. пререгистрировать полный факторный эксперимент  $2 \times 2$ : топология вызовов (раздельная/объединённая)  $\times$  отбор контекста (выключен/включён);
2. использовать не менее 149 уникальных пар без вредных исходов — предпочтительно все 164 HumanEval и несколько seed — для точной 2-п.п. цели неухудшения;
3. расширить сетку ниже 256 токенов и включить области выигрыша и проигрыша до оценки crossover или routing gate;
4. сохранять полный вход, выход, хеш промпта, точный идентификатор снимка модели и результат внешнего теста для каждого запуска;
5. заменить приближённый SWE-bench-путь официальным контейнерным harness и провести оценку на случайной выборке репозиторий;
6. повторить E2c на задачах с независимыми реальными инструментами и сопоставить последовательный и параллельный режимы при одинаковой модели;
7. сравнить B3 с Self-Refine, SPP, Self-Collaboration, S2-MAD, Agentless, ReAct и SWE-agent при совпадающих вызовах, контексте и проверке;

8. реализовать E4 как рандомизированный либо парный policy-эксперимент с сохранением каждого решения screening, перезапуска, финального кандидата и стоимости;
9. измерить полный ТСО, включая труд сопровождения, задержку, число повторных запусков, инфраструктуру и хранение, на производственном распределении длины контекста.

## 11 Заключение

Hybrid-INoT оценён как интегрированная промежуточная система между внутри-контекстными ролями и внешней мультиагентной оркестрацией. В двух реальных API-пилотах пакет В3 использовал на 49.6–90.3% меньше токенов, чем сопоставленный В2; четырёхточечная Flash-Lite OLS-сводка равна 3.425 для В2 и 0.219 для В3 токена на токен контекста. Каждая наблюдаемая Flash-Lite-пара превысила ресурсный порог 15%, а условный rank-based анализ чувствительности прошёл поправку на множественные сравнения, но полная Н1 не доказана: 20 потолочных неслучайных пар недостаточно для допуска неухудшения 2 п.п. Н2 не оценена агрегированным screening-harness, Н3 не поддержана; маршрутизация и атрибуция компонентов остаются открытыми. \$429.96 — экономия API-инференса для фиксированной смеси, а не популяционный ТСО. Защищаемый вывод состоит в том, что полный пакет В3 уменьшает повторную передачу длинного контекста в этих контролируемых наборах; качество, причинный вклад компонентов, польза маршрутизатора и эффект на реальных репозиториях требуют мощного пререгистрированного исследования.

## Доступность данных и кода

Код исследования, конфигурации и исходные результаты доступны в неизменяемом commit, указанном в [34]. Репозиторий статьи содержит входы репликации Flash-Lite, SHA-256 manifest, переносимый evidence snapshot и скрипт построения рисунков. Скрипт пересчитывает из строк E3 агрегаты, парные диагностики, контекстные сводки, агрегированную E4 screening-таблицу и E5-экстраполяцию; готовые E2/E6-сводки импортируются с хешами, а не пересчитываются. Полные сгенерированные ответы в сохранённом `runs.json` отсутствуют, поэтому pass/fail labels и исходные внешние тесты нельзя независимо воспроизвести только из артефакта статьи. Эти границы и хеш аналитического скрипта записаны в evidence snapshot.

## Декларации

**Финансирование:** внешнее финансирование не заявлено. **Конфликт интересов:** автор заявляет об отсутствии конфликта интересов. **Этика:** использованы публичные программные бенчмарки; люди и персональные данные в исследовании не участвуют. **Вклад автора:** Д.П. спроектировал исследование, реализовал системы, провёл эксперименты, аудировал свидетельства и подготовил рукопись.

## References

- [1] H. Sun and S. Zeng, “Introspection of Thought Helps AI Agents,” arXiv preprint arXiv:2507.08664, 2025.
- [2] A. J. Ko, R. DeLine, and G. Venolia, “Information Needs in Collocated Software Development Teams,” in *Proc. 29th Int. Conf. Software Engineering*, 2007, pp. 344–353.
- [3] C. Parnin and S. Rugaber, “Resumption Strategies for Interrupted Programming Tasks,” *Software Quality Journal*, vol. 19, no. 1, pp. 5–34, 2011.
- [4] M. K. Goncalves, C. R. B. de Souza, and V. M. Gonzalez, “Collaboration, Information Seeking and Communication,” *J. Universal Computer Science*, vol. 17, no. 11, pp. 1529–1550, 2011.
- [5] J. Wei *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *NeurIPS*, vol. 35, 2022, pp. 24824–24837.
- [6] S. Yao *et al.*, “ReAct: Synergizing Reasoning and Acting in Language Models,” *ICLR*, 2023.
- [7] A. Madaan *et al.*, “Self-Refine: Iterative Refinement with Self-Feedback,” *NeurIPS*, 2023.
- [8] N. Shinn *et al.*, “Reflexion: Language Agents with Verbal Reinforcement Learning,” *NeurIPS*, 2023.
- [9] Z. Wang *et al.*, “Unleashing Cognitive Synergy in Large Language Models: A Task-Solving Agent through Multi-Persona Self-Collaboration,” in *Proc. NAACL-HLT*, 2024, pp. 257–279, doi:10.18653/v1/2024.naacl-long.15.
- [10] Y. Dong, X. Jiang, Z. Jin, and G. Li, “Self-Collaboration Code Generation via ChatGPT,” *ACM Trans. Software Engineering and Methodology*, vol. 33, no. 7, art. 189, pp. 1–38, 2024, doi:10.1145/3672459.
- [11] Z. Zeng *et al.*, “S2-MAD: Breaking the Token Barrier to Enhance Multi-Agent Debate Efficiency,” in *Proc. NAACL-HLT*, 2025, pp. 9393–9408, doi:10.18653/v1/2025.naacl-long.475.
- [12] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Agentless: Demystifying LLM-based Software Engineering Agents,” arXiv:2407.01489, 2024.
- [13] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, “Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting,” *NeurIPS*, 2023.
- [14] G. Weiss, *Multiagent Systems*. MIT Press, 1999.

- [15] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Wiley, 2009.
- [16] S. Hong *et al.*, “MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework,” *ICLR*, 2024.
- [17] C. Qian *et al.*, “ChatDev: Communicative Agents for Software Development,” *ACL*, 2024.
- [18] T. Guo *et al.*, “Large Language Model based Multi-Agents: A Survey of Progress and Challenges,” arXiv:2402.01680, 2024.
- [19] C. Qian *et al.*, “Scaling Large Language Model-based Multi-Agent Collaboration,” arXiv:2406.07155, 2024.
- [20] C. E. Jimenez *et al.*, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” *ICLR*, 2024.
- [21] J. Yang *et al.*, “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” *NeurIPS*, 2024.
- [22] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code,” arXiv:2107.03374, 2021.
- [23] J. Austin *et al.*, “Program Synthesis with Large Language Models,” arXiv:2108.07732, 2021.
- [24] H. Jiang *et al.*, “LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression,” *ACL*, 2024.
- [25] Z. Pan *et al.*, “LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression,” *ACL Findings*, 2024.
- [26] H. Jia, E. T. Barr, and S. Mehtaev, “Compressing Code Context for LLM-based Issue Resolution,” arXiv:2603.28119, 2026.
- [27] Z. Bi *et al.*, “Iterative Refinement of Project-Level Code Context for Precise Code Generation with Compiler Feedback,” arXiv:2403.16792, 2024.
- [28] C.-Y. Su and C. McMillan, “Do Code LLMs Do Static Analysis?,” arXiv:2505.12118, 2025.
- [29] M. Sepidband, H. Taherkhani, S. Wang, and H. Hemmati, “Enhancing LLM-Based Code Generation with Complexity Metrics,” arXiv:2505.23953, 2025.
- [30] ISO/IEC 25010:2023, “Systems and software engineering — SQuaRE — Product quality model,” ISO, 2023.



- [31] T. J. McCabe, “A Complexity Measure,” *IEEE Trans. Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.
- [32] F. Wilcoxon, “Individual Comparisons by Ranking Methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [33] S. Holm, “A Simple Sequentially Rejective Multiple Test Procedure,” *Scandinavian J. Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [34] D. Privezentsev, “INoT\_Research: reproducible experiments for Hybrid-INoT,” GitHub repository, commit 010211ee5775185e8330ab6cde06e912c2adcb9e, 2026. [Online]. Available: [https://github.com/daniil-novel/INoT\\_Research](https://github.com/daniil-novel/INoT_Research). Accessed: 28 July 2026.
- [35] OpenRouter, “Gemini 3.1 Pro Preview — pricing,” 2026. [Online]. Available: <https://openrouter.ai/google/gemini-3.1-pro-preview/pricing>. Accessed: 26 July 2026.
- [36] OpenRouter, “Gemini 3.1 Flash Lite Preview — pricing,” 2026. [Online]. Available: <https://openrouter.ai/google/gemini-3.1-flash-lite/pricing>. Accessed: 26 July 2026.